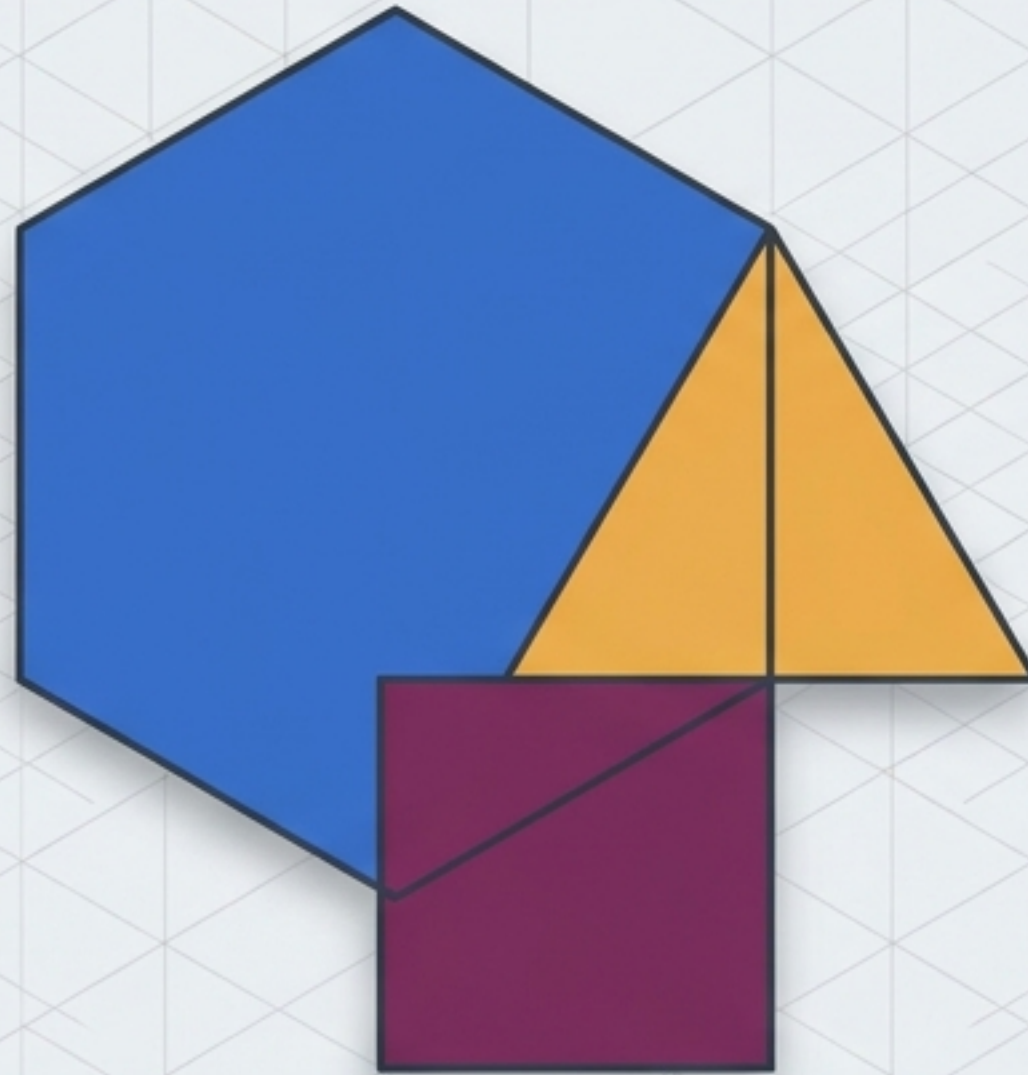
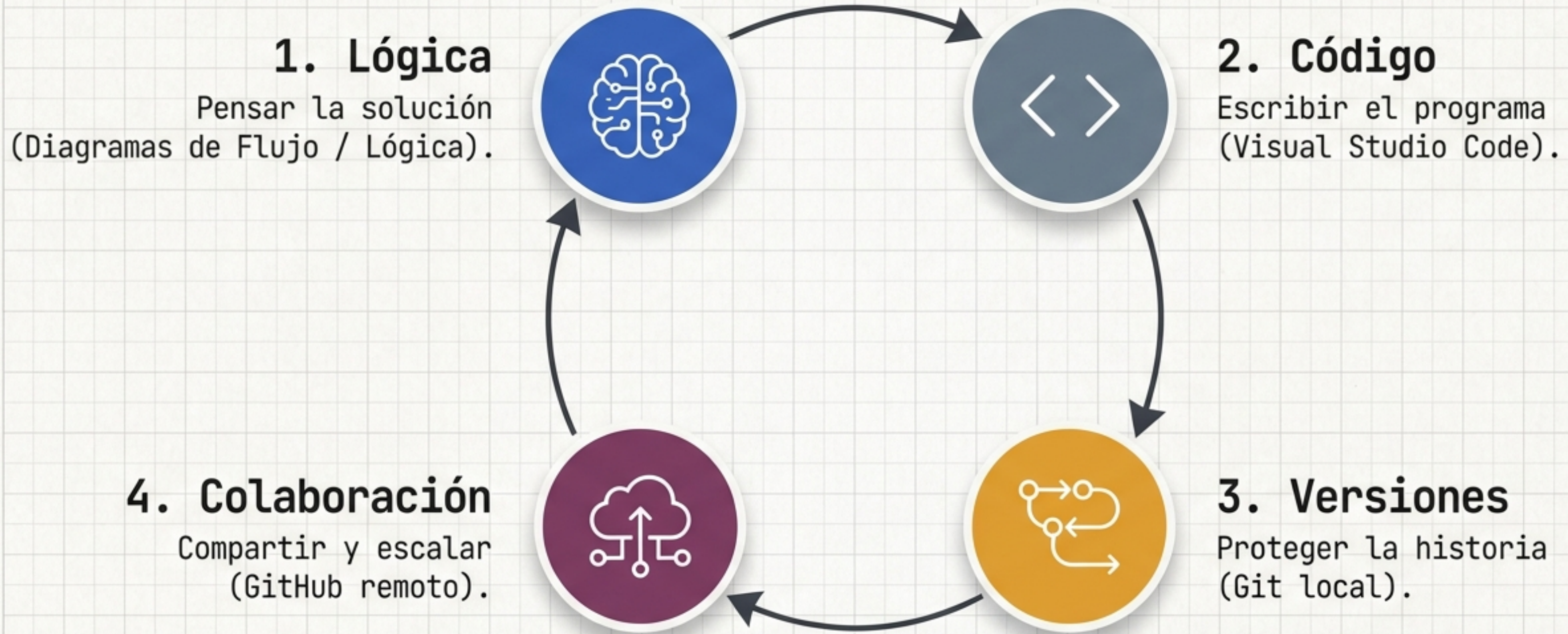


El Ecosistema del Programador

Herramientas y prácticas esenciales para el desarrollo de software moderno.



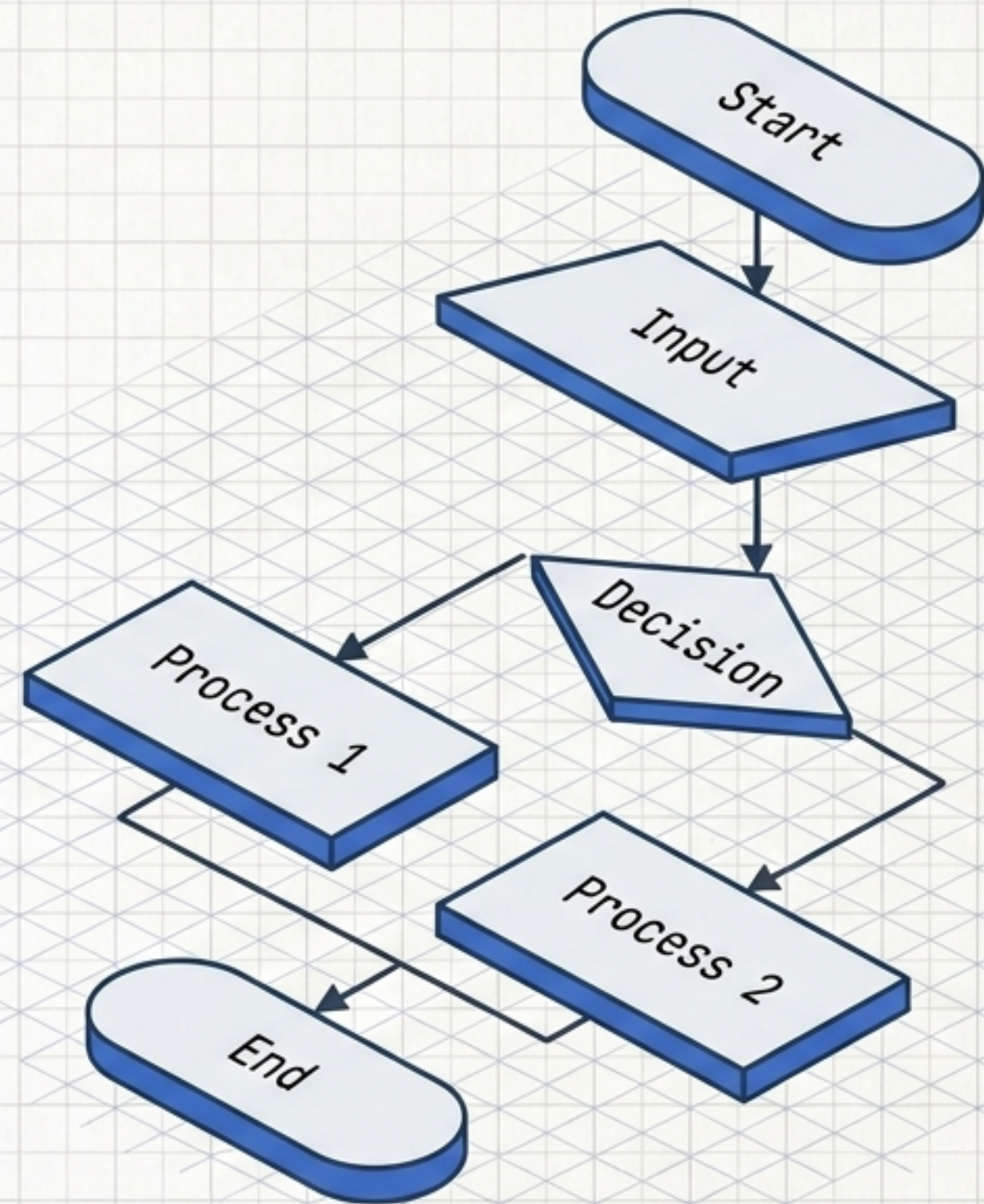
El ciclo de vida del código



Antes del código, existe la lógica

Las herramientas como los Diagramas de Flujo de Datos (DFD) o el pseudocódigo (PSeInt) permiten modelar la solución independientemente del lenguaje de programación.

La programación secuencial, condicional y cíclica es universal. Dominar la lógica previene errores estructurales antes de escribir la primera línea de código.



Código limpio: Escribir para humanos

El código se lee mucho más de lo que se escribe. – Ward Cunningham (Inventor de la Wiki).

1. ¿Por qué existe?
2. ¿Qué hace?
3. ¿Cómo se usa?

INCORRECTO

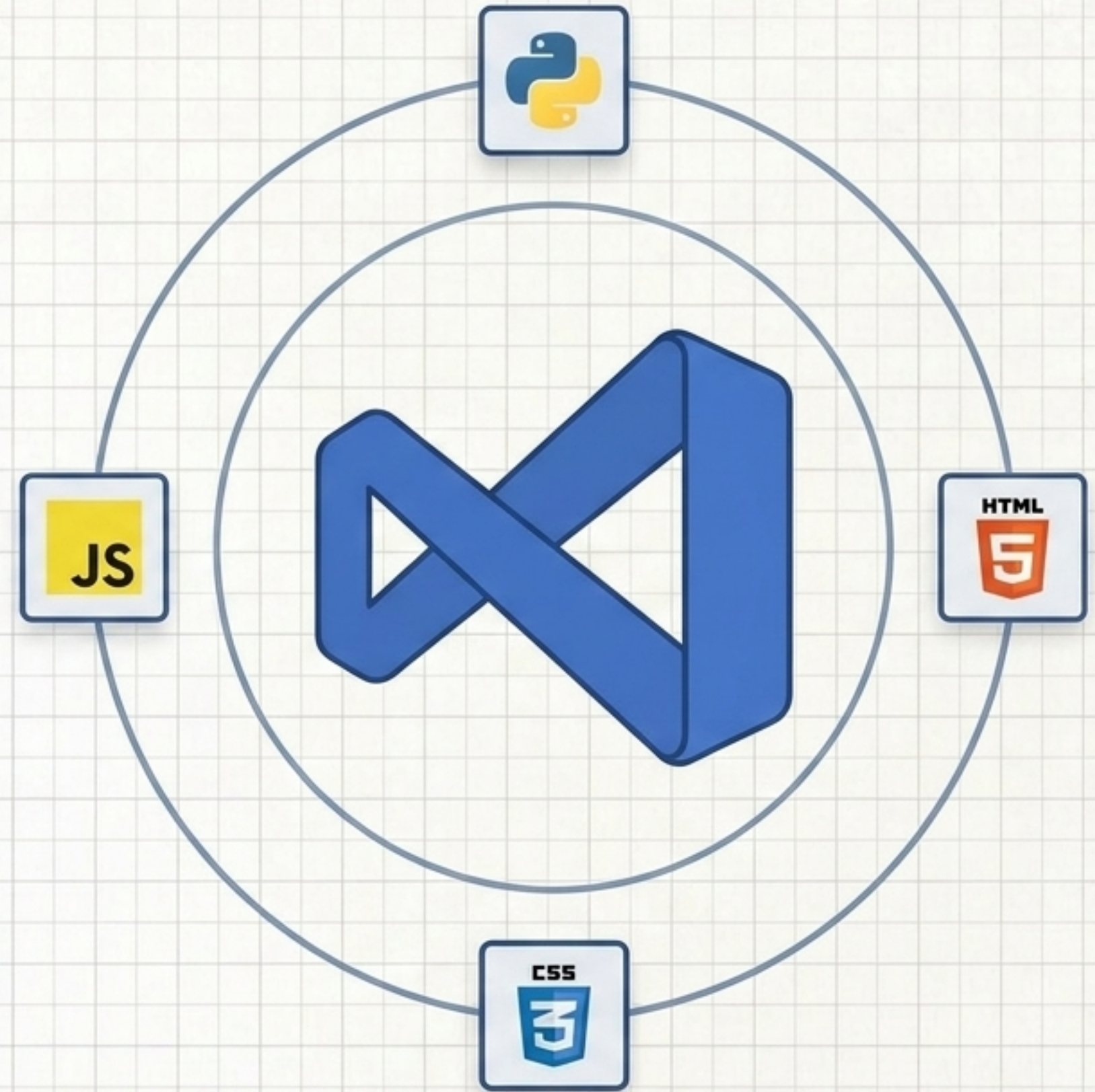
```
1 | x = 5
2 | y = 10
3 | data1 = x + y
```

CORRECTO

```
1 | nombre_aspirante = 5
2 | disparos_atajados = 10
3 | puntaje_total = nombre_aspirante
   | + disparos_atajados
```


El taller principal: Visual Studio Code

El editor de código fuente definitivo. Creado por Microsoft, es ligero, altamente personalizable a través de extensiones y adoptado universalmente por la industria. No es un IDE pesado, sino un entorno modular que crece según las necesidades del proyecto.

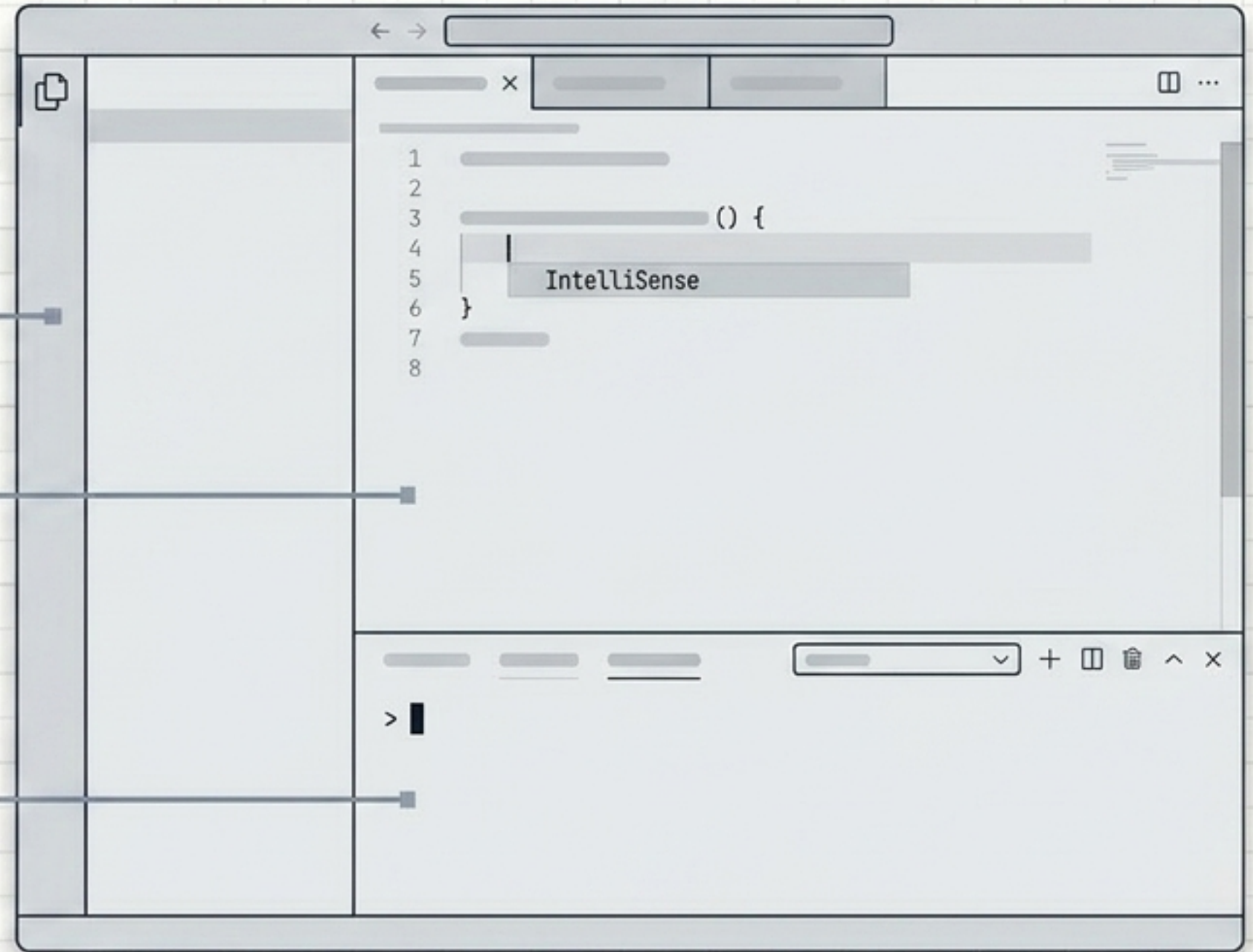


Anatomía del espacio de trabajo

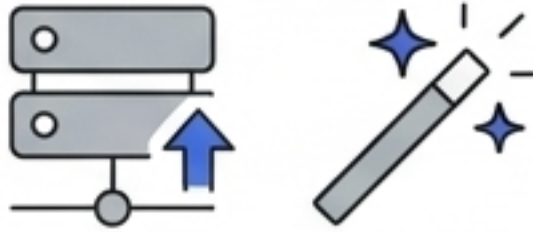
Activity Bar / Explorer: Navegación de archivos, carpetas, extensiones y control de código fuente.

Editor Area: El lienzo principal donde ocurre la codificación, con soporte para pestañas múltiples e IntelliSense.

Integrated Terminal: Ejecución directa de comandos de sistema y Git sin abandonar el editor.



Superpoderes del Editor

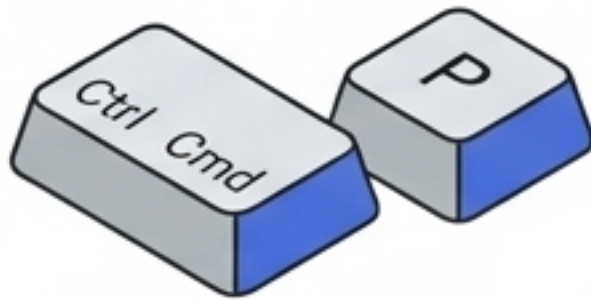


Extensiones clave: Live Server (vista previa en tiempo real) y Prettier (formateo automático de código).

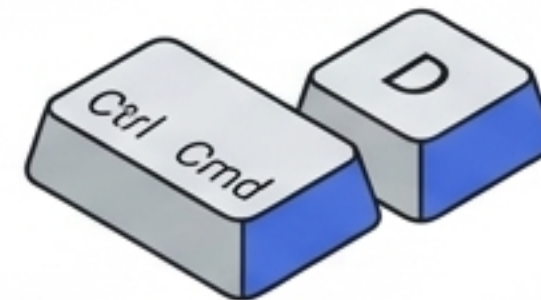


> Command Palette (Cmd/Ctrl + Shift + P)

El buscador centralizado para ejecutar cualquier acción o configuración.



Navegación Rápida: (Cmd/Ctrl + P).
Búsqueda y apertura instantánea de archivos en proyectos grandes.

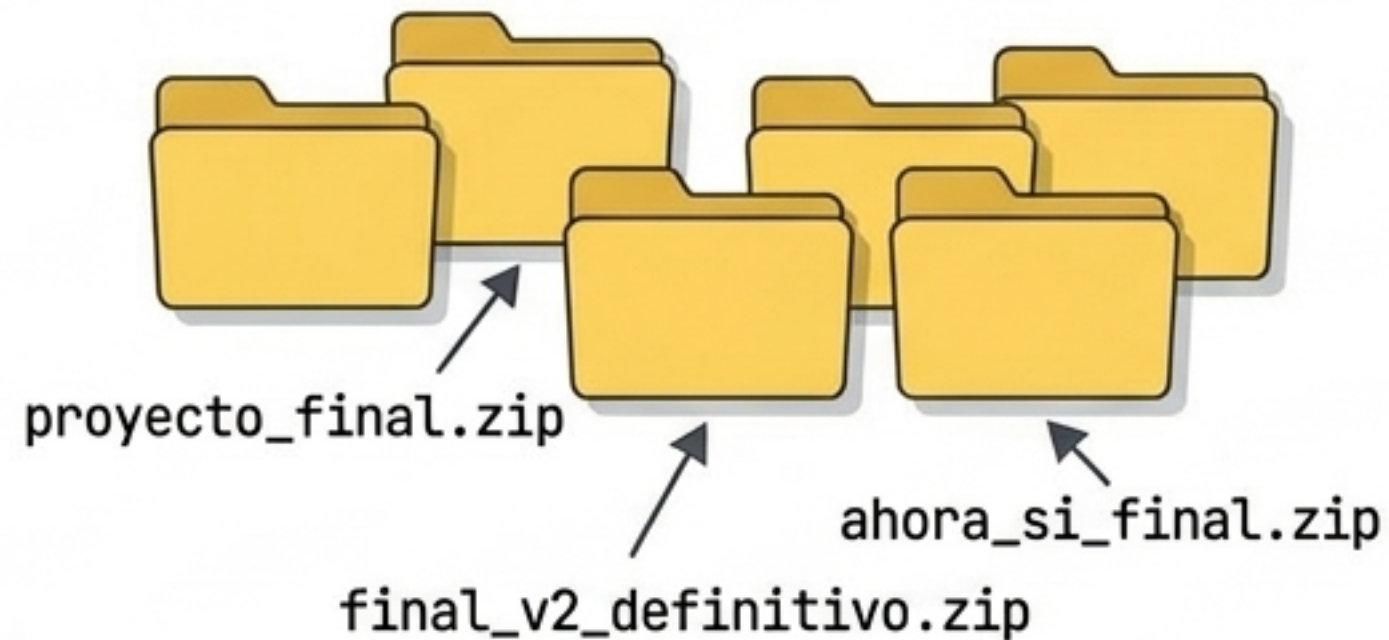


Cursores Múltiples: (Cmd/Ctrl + D).
Seleccionar y editar múltiples instancias de la misma palabra simultáneamente.

El problema del control de versiones

Creado por Linus Torvalds en 2005 para gestionar el kernel de Linux, Git es un sistema de control de versiones distribuido. Elimina el caos de duplicar carpetas, creando una línea de tiempo inalterable y precisa del historial del proyecto.

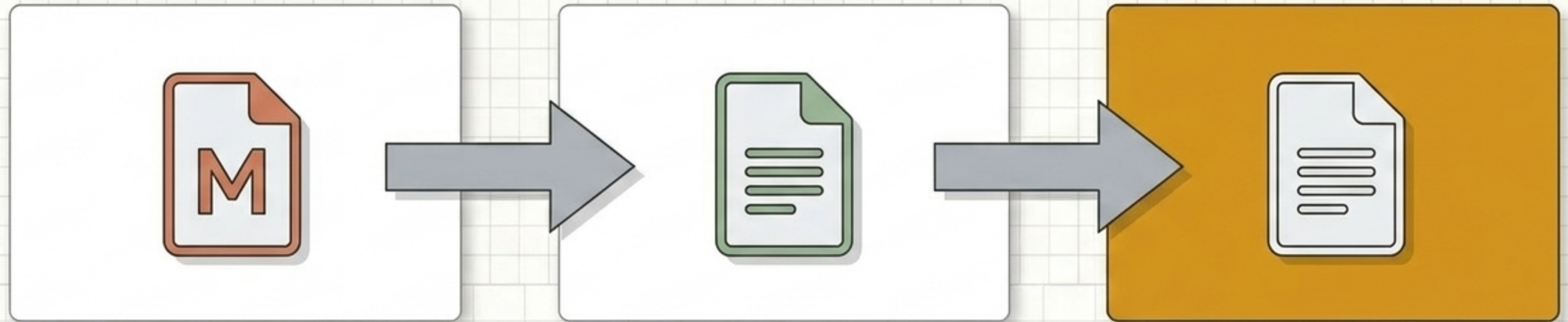
The Problem



The Solution



Los tres estados de Git



1. Working Directory

El taller. Donde creas, editas y modificas archivos.

2. Staging Area

El muelle de carga. Seleccionas qué archivos están listos para ser guardados usando `git add`.

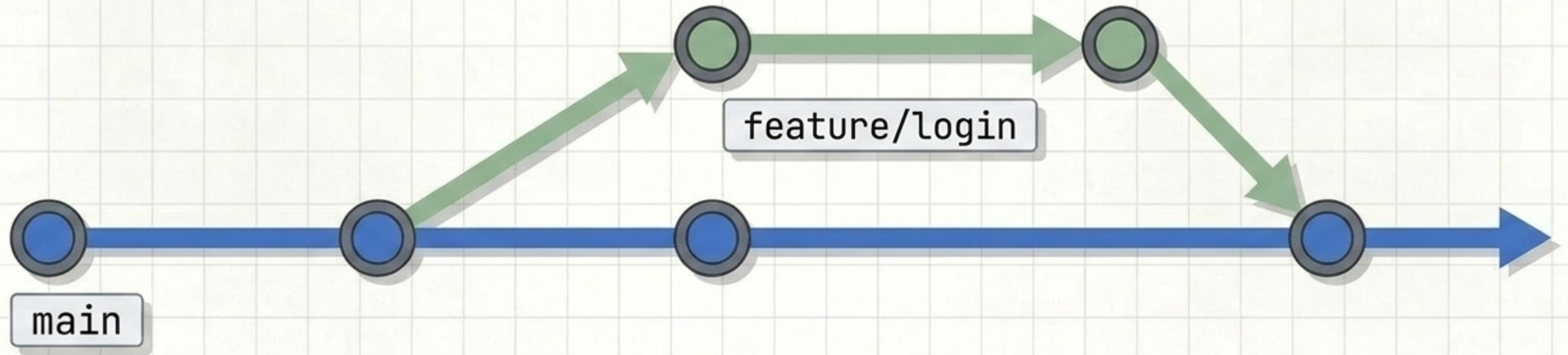
3. Repository

La bóveda. Guardas una instantánea permanente (commit) en la historia del proyecto usando `git commit -m`.

Ramas (Branches) y Fusiones (Merges)

Las ramas permiten aislar el desarrollo de nuevas características o correcciones.

Múltiples desarrolladores pueden trabajar en paralelo en sus propias ramas sin afectar el código de producción en la rama principal (main). Al finalizar, los cambios se fusionan (**git merge**).



La máquina del tiempo: Redes de seguridad

`git checkout / git switch`

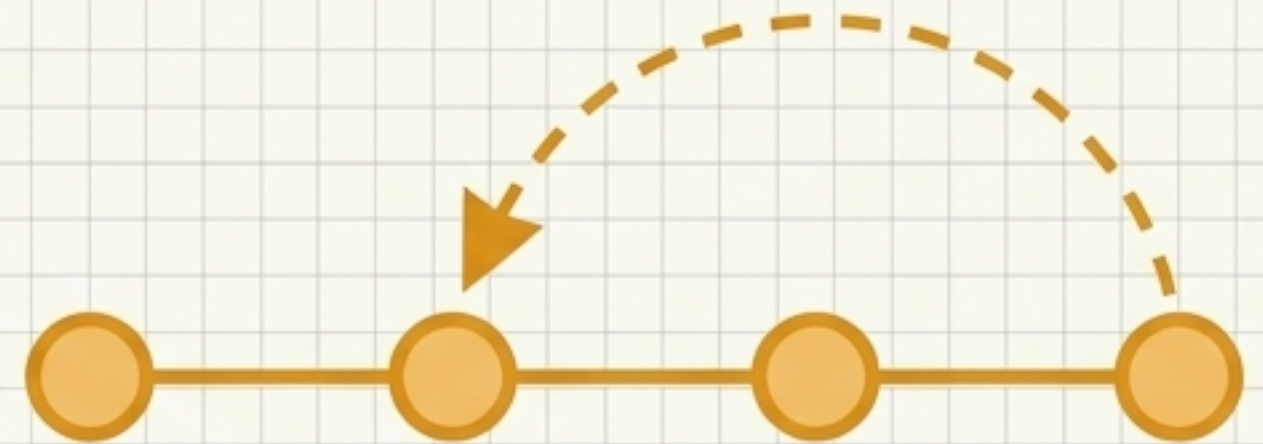
Permite viajar en el tiempo a versiones anteriores del código o saltar entre diferentes ramas activas.

`git reset --hard`

Una acción destructiva para volver a un punto anterior, descartando todos los cambios recientes.

`git stash`

Guarda temporalmente el trabajo en progreso que aún no está listo para un commit, permitiendo cambiar de rama con un área de trabajo limpia.

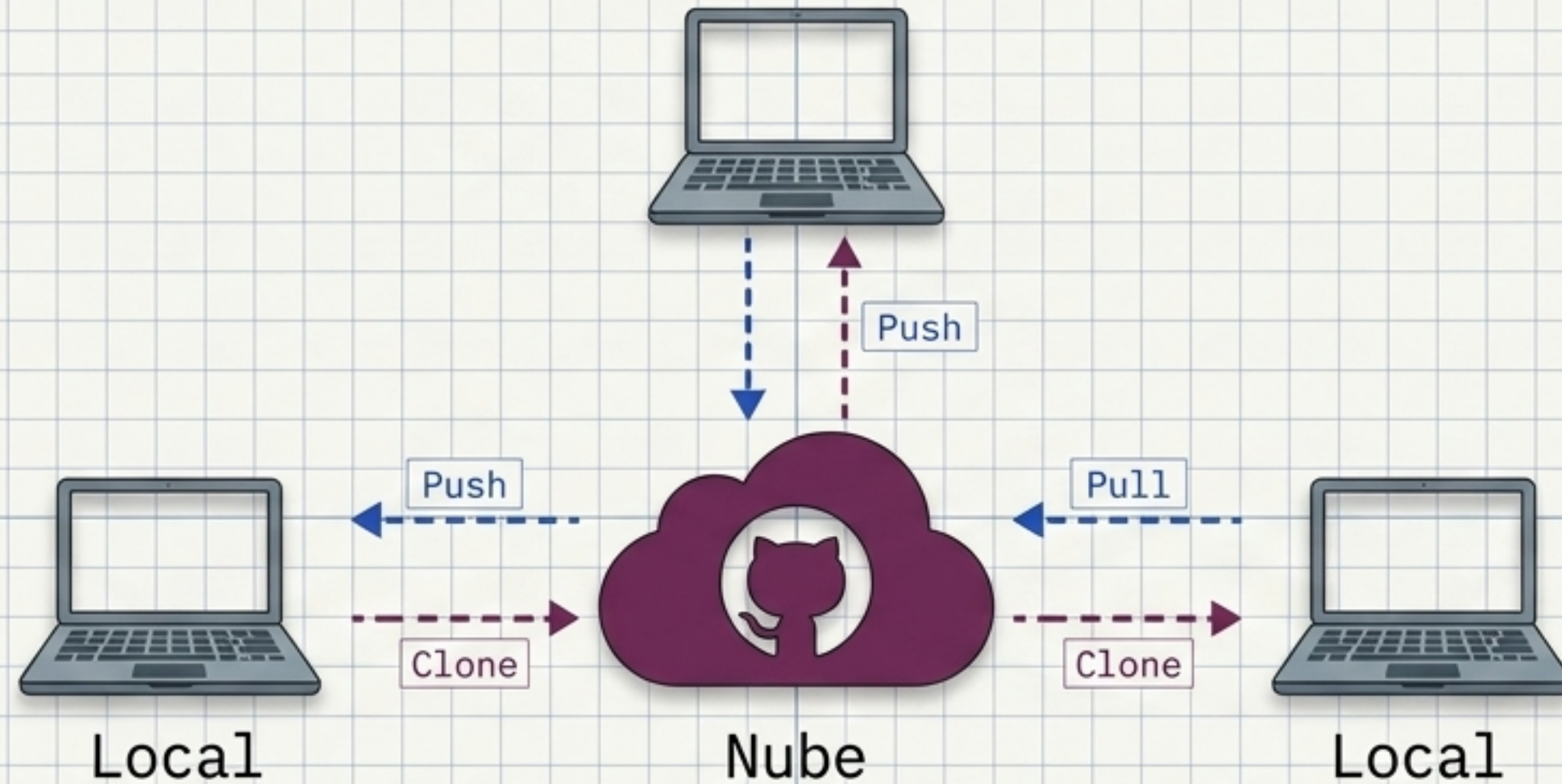


Git vs. GitHub: Aclarando la confusión

		
Naturaleza	Motor local de control de versiones.	Plataforma en la nube de alojamiento de código.
Interfaz	Principalmente interfaz de línea de comandos (Terminal).	Interfaz web gráfica y visual.
Propósito	Rastrear el historial y los cambios de los archivos.	Facilitar la colaboración asíncrona, revisión de código y código abierto.

Más del 90% de los desarrolladores profesionales utilizan esta combinación a diario.

Sincronización con la nube



Clone

Descargar una copia exacta de un repositorio remoto a tu máquina local.

Push / Pull

'Push' empuja tus **commits locales** al servidor de GitHub.
'Pull' trae los últimos cambios del equipo a tu máquina.

Pull Request (PR)

El corazón de la colaboración. Proponer formalmente que tus cambios sean revisados e integrados al proyecto principal.

Interacción: Terminal vs. Interfaz Gráfica (GUI)

CLI (Terminal)

```
$ git status
On branch main
Your branch is up to date with 'origin/main'.

nothing to commit, working tree clean

$ git add .
$ git commit -m "feat: implement login logic"
[main 4f5e2a1] feat: implement login logic
2 files changed, 15 insertions(+), 3 deletions(-)
```

El lenguaje nativo. Ofrece control absoluto, velocidad, y es el estándar universal en cualquier servidor.

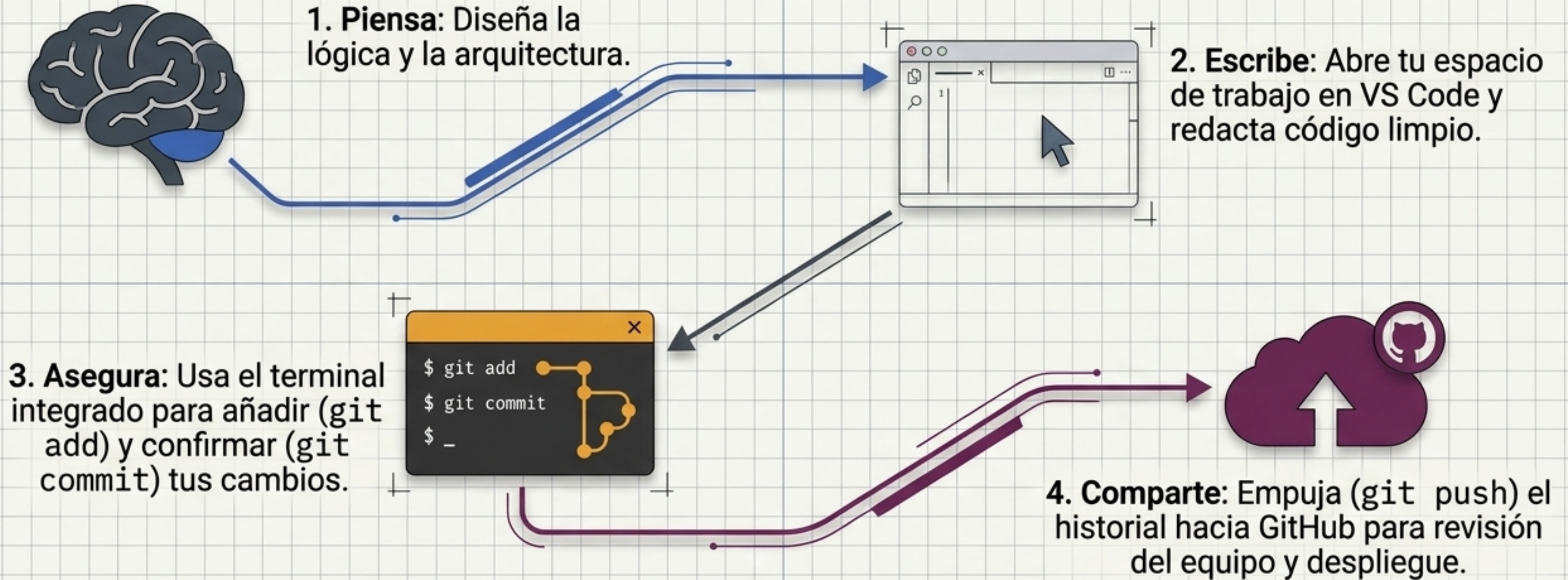
GUI (GitKraken / GitHub Desktop)



Red de seguridad visual. Facilita la comprensión de árboles de ramas complejos y la resolución de conflictos sin tener que memorizar comandos.

Ambas son herramientas profesionales. Comprender la terminal te da los cimientos; usar una GUI te da velocidad visual.

Un día en la vida: El ecosistema unificado



La fluidez técnica no proviene de conocer todas las herramientas, sino de dominar cómo se conectan entre sí.